

Ghost Identity Hunter — File-by-File Reference

Detailed per-file documentation of what each source file does and how it works. Companion to [ARCHITECTURE.md](#) (system view) and [LLD.md](#) (subsystem design).

Artifact dict convention used throughout the pipeline:

```
{"type", "value", "source", "confidence", "link_type", "metadata"}
```

Table of Contents

1. [Investigation flow \(how files connect\)](#)
 2. [Entry points](#)
 3. [Orchestration](#)
 4. [Configuration](#)
 5. [Storage](#)
 6. [OSINT modules](#)
 7. [Plugin subsystem](#)
 8. [Built-in plugins](#)
 9. [Correlation](#)
 10. [Reporting and visualization](#)
 11. [Utilities](#)
 12. [Analysis and collaboration](#)
 13. [API](#)
 14. [Config, scripts, packaging, Docker](#)
 15. [Tests](#)
-

Investigation flow (how files connect)

```
CLI / API
|
▼
orchestrator.run_investigation
| BFS by depth level
├─ modules/*           (phone, email, username, image, breach, dorks, leakosint)
├─ modules/external_tools.py (subprocess OSINT CLIs)
│   └─ modules/tool_parsers.py (one parser per tool's own format)
├─ plugins/manager.py → builtins/*
├─ storage/evidence.py (hashed raw-output captures)
└─ storage/database.py (artifacts, links, presence, evidence)
    |
    ▼
correlation/linker.py + scorer.py
    |
    ▼
reporting/report_data.py (derived sections, redaction)
    |
    ▼
reporting/html_report.py + templates/standard.html + graph/visualizer.py
```

1. Entry points

`src/__init__.py`

Package marker for Ghost Identity Hunter. Docstring only; no runtime logic.

`src/cli.py`

Purpose: Click CLI entrypoint (`ghost-hunter` / `python -m src.cli`).

Commands:

Command	Role
<code>investigate</code>	Build seeds → <code>InvestigationConfig</code> → <code>run_investigation</code> → optional auto-report
<code>report</code>	HTML / JSON / PDF / CSV report for an investigation ID (<code>--template</code> , <code>--sections</code> , <code>--redact</code> , <code>--compare</code>)
<code>graph</code>	Interactive pyvis identity graph HTML (<code>--layout</code> , <code>--collection-threshold</code>)
<code>list</code>	List investigations in the SQLite DB
<code>correlate</code>	Print identity profiles and risk scores
<code>evidence</code>	Re-hash every preserved capture against its recorded digest; exits non-zero if anything was altered
<code>plugins</code> <code>list info enable disable</code>	Plugin discovery / in-memory enable toggles

How it works:

1. Root `cli` group stores `--verbose` / `--db` on Click context and configures logging (`setup_logging` → console + `logs/<user>_<timestamp>.log`).
2. `investigate` requires at least one seed (`-p/-e/-u/-n/-i/-d/--ip`) or `--check-tools` . Seeds become typed dicts (`phone` , `email` , `username` , `fullname` , `image` , `domain` , `ip_address`).
3. Opens SQLite via `get_connection` , runs the orchestrator, prints summary stats, optionally generates reports (`--report-format html|json|both`).
4. Other commands load an investigation by ID and call linker, visualizer, or report generators.
5. A run that stops early raises `InvestigationAborted` ; the CLI catches it, summarises what was stored from the DB and still generates the report, rather than printing a traceback over the findings.

Key helpers: `_json_output_path` (avoid HTML/JSON path collision), `_partial_result` (counts for an aborted run), `_print_tool_coverage` (integrated vs available tools).

Report flags: `--report-format html|json|both|pdf|csv` , `--report-template standard|executive|technical|legal` , `--report-sections` , `--redact-report` , `--strict-match/--no-strict-match` .

Calls: `orchestrator` , `storage.database` , `correlation.linker/scorer` , `graph.visualizer` , `reporting.html_report` , `external_tools.get_tool_coverage` , `tool_checker` , `plugins` , `config.loader` .

2. Orchestration

`src/orchestrator.py`

Purpose: Depth-limited, level-parallel BFS investigation engine. Central dispatcher for modules, external tools, and plugins.

Key types:

Symbol	Role
InvestigationConfig	Depth, breach/username/image toggles, Neo4j, Google Dorks, runtime/artifact budgets, worker count
InvestigationResult	Investigation id, counts, risk indicators
InvestigationAborted	Raised when a run stops early, naming the investigation so the caller can still report on it
_safe_correlation	Graph metrics, or bare counts if the analysis fails — never the run
ArtifactProcessResult	Deferred DB writes: <code>discovered</code> , <code>source_metadata</code> , <code>platform_presences</code>
run_investigation	Main pipeline
_process_artifact	Type dispatcher
_process_phone/_email/_username/_fullname/_image	Native module paths
_process_external_tools / _run_external_tool_tasks	Concurrent CLI tool runs
_process_with_plugins	PluginManager execution
_username_candidates	Full-name → username variants
_account_presences	Normalize account findings into presence rows

How it works:

1. Creates an investigation row; enqueues seed artifacts at depth 0.
2. Processes **by BFS level**: workers run `_process_artifact` (network/subprocess only — no SQLite writes from workers).
3. Main thread applies metadata/presence updates, dedups via `seen[type:value]`, adds links (including to already-seen nodes), and re-queues only expandable types up to `max_depth` / `max_total_artifacts`.
4. Wall-clock budget (`max_runtime_minutes`) enforced at level boundaries and mid-level via `as_completed(timeout=...)`.
5. After BFS: `complete_investigation`, NetworkX or Neo4j correlation into `investigation_metadata`, risk indicators harvested from artifact metadata.
6. Findings are stored as they are made and every stage after the BFS is guarded, so nothing a tool or module does can cost the run its report — worker exceptions are logged per artifact, correlation failure degrades to counts, and any other failure surfaces as `InvestigationAborted`.
7. Findings are filtered by `utils/matching` when strict matching is on, so a tool result that does not carry the target's exact handle is dropped rather than attributed.

Artifact routing:

Type	Native module	External tools	Plugins
phone	phone_osint	—	phone_validation, leakosint
email	email_osint + breach_check	holehe	email_breach, holehe, leakosint
username	username_search (+ profile image scrape)	sherlock, maigret, osrframework, google_dorks	matching builtins, leakosint
fullname	image_match + username variants	google_dorks	image_match (if registered)
image	image_search	exiftool	exiftool
domain / subdomain	—	whois, subfinder, amass, whatweb, wayback, theHarvester	matching builtins
ip_address	—	nmap, shodan	matching builtins
platform_presence	—	—	profile_image

Config keys: `orchestrator.{bfs_batch_size,max_depth,max_parallel_workers}`, `investigation.{max_runtime_minutes,max_total_artifacts}`, `plugin_settings.max_parallel_workers`.

Called by: `cli.investigate`, `api.workflow_api`.

3. Configuration

`src/config/__init__.py`

Re-exports `ConfigLoader`, `get_config`.

`src/config/loader.py`

Purpose: YAML load/save with dotted-key get/set and plugin helpers.

How it works: Searches `./config/config.yaml`, `./config.yaml`, `~/.ghosthunter/config.yaml`, etc. Falls back to embedded defaults. Dot-notation walks nested dicts. Plugin config merges `plugin_settings` globals with per-plugin `plugins.<name>` overrides. Process-global singleton via `get_config` / `reload_config`.

Key API: `load_config`, `save_config`, `get/set`, `get_plugin_config`, `set_plugin_enabled`, `list_plugins`, `get_enabled/disabled_plugins`.

4. Storage

`src/storage/__init__.py`

Package docstring only.

`src/storage/database.py`

Purpose: SQLite persistence for investigations, artifacts, links, platform presence, metadata, and audit trail.

Key API:

Function	Role
<code>get_connection</code>	WAL + FKs + schema init; default <code>~/.ghost_hunter/investigations.db</code>
<code>create_investigation</code> / <code>complete_investigation</code> / <code>get_investigation</code> / <code>list_investigations</code>	Investigation lifecycle
<code>add_artifact</code> / <code>add_artifacts_bulk</code> / <code>get_artifacts</code>	Dedupe on <code>(investigation_id, type, value)</code>
<code>add_link</code> / <code>get_links</code>	Dedupe on source/target pair
<code>add_platform_presence</code> / <code>get_platform_presences</code>	Platform account rows (<code>is_verified</code>)
<code>add_audit_log</code> / <code>get_audit_trail</code>	Audit helpers (write path little-used today)
<code>add_evidence</code> / <code>get_evidence</code>	Preserved raw-output captures (path, sha256, byte size, tool, command)
<code>_add_missing_columns</code>	Idempotent additive migration for older databases

How it works: `sqlite3.Row` factory, WAL journaling, foreign keys on. Schema is idempotent `CREATE TABLE IF NOT EXISTS`. IDs are prefixed short UUIDs: `INV-`, `ART-`, `LNK-`, `PRS-`, `AUD-`. Artifact `metadata` is JSON text.

Tables: `investigations`, `artifacts`, `artifact_links`, `platform_presence`, `investigation_metadata`, `audit_trail`, `evidence` (+ `comments` created by collaboration module).

`src/storage/evidence.py`

Purpose: Preserve the raw output behind every finding, hashed, so a report can be defended rather than merely asserted.

How it works: `begin(investigation_id)` opens a capture session, `record(...)` writes each tool's stdout or response body to `<directory>/<INV>/<sha256>.txt` with its digest, byte size, command line, duration, exit status and `tool_version`, and

`flush(conn)` writes the rows. The `analysing(operation, target)` context manager labels captures taken deep inside an integration with what asked for them. Captures are truncated at `MAX_CAPTURE_BYTES` (2 MB) with a notice, and written `0700 / 0600` since they hold subject data. `verify(rows)` re-hashes stored files and reports `verified / modified / missing` — what `ghost-hunter evidence` prints.

Caveat: the session is process-global, so two investigations run concurrently in one process (the HTTP API, not the CLI) share it. Captures also accumulate indefinitely; there is no retention policy yet.

Config: `evidence.{enabled,directory}`.

5. OSINT modules

`src/modules/__init__.py`

Package docstring only.

`src/modules/phone_osint.py`

Purpose: Validate phone numbers; classify carrier, line type, VoIP/burner risk via `phonenumbers`.

How it works: Parse with region fallback to US → validate → fill E.164/national formats, country, timezone, carrier, line type. Flags VoIP (number type or known carrier list) and disposable services as `risk_indicators`. Invalid formats still produce risk-only results.

Key types: `PhoneAnalysis` (+ `to_json`), `analyze_phone`, `get_discovered_artifacts` (optional `carrier_info`).

Sets: `VOIP_CARRIERS`, `DISPOSABLE_SERVICES`.

`src/modules/email_osint.py`

Purpose: Email format/domain risk classification plus Gravatar / GitHub / HIBP enrichment.

How it works: Validate format → split local/domain → flag disposable/privacy domains → treat non-free non-disposable as corporate. For enrichment targets: parallel Gravatar HEAD, GitHub user-search-by-email, HIBP truncated breach list. Emits usernames discovered via platforms as artifacts.

Key API: `EmailAnalysis`, `validate_email_format`, `check_gravatar`, `check_github_email`, `check_hibp_breaches`, `analyze_email`, `get_discovered_artifacts`.

Config: `email_osint.{max_parallel_workers,disposable_domains,privacy_providers}`.

`src/modules/username_search.py`

Purpose: Parallel multi-platform username existence checks with content/API validation to reduce soft-404 false positives.

How it works:

1. Loads platform list from `username_search.platforms` in config (GitHub/GitLab/Reddit APIs + content-validated web platforms).
2. Concurrent HTTP checks via shared session.
3. Web platforms require failure/success markers and/or username-in-body; failure markers matched **before** success markers.
4. Validation strengths `content|api|status` map to confidence $\approx 0.9 / 0.9 / 0.4$.
5. In-memory cache (~1000 entries). Emits `platform_presence` artifacts.

Key API: `PlatformResult`, `UsernameSearchResult`, `_check_platform`, `_validate_web_content`, `search_username`, `search_usernames_batch`, `generate_username_variants`, `get_discovered_artifacts`.

`src/modules/breach_check.py`

Purpose: HaveIBeenPwned email breach lookup (API key) or mock demo data; free Pwned Passwords k-anonymity check.

How it works: With API key → HIBP v3 `breachedaccount`. Without key → synthetic Adobe/LinkedIn/Facebook-style demo breaches. Risk from breach count + password exposure + data classes. Password check sends SHA-1 **prefix only**.

Key API: `BreachInfo`, `BreachCheckResult`, `check_email_breaches`, `check_password_exposure`, `get_discovered_artifacts`.

Config: `breach_check.{hibp_api_base,rate_limit_seconds}`.

`src/modules/image_search.py`

Purpose: Local image EXIF/GPS/hash analysis and reverse-search URL generation.

How it works: Pillow reads EXIF/GPS → MD5/SHA-256 file hashes → builds Google/TinEye/Yandex/Lens URLs → emits location/camera/risk artifacts (`gps_coordinates`, `camera_info`, `no_exif_metadata`, etc.).

Key API: `ExifData`, `ImageAnalysis`, `extract_exif`, `compute_file_hashes`, `generate_reverse_search_urls`, `analyze_image`, `get_discovered_artifacts`.

`src/modules/image_match.py`

Purpose: Name-based image search + optional face encodings; profile-image URL scraping.

How it works: Search images for a full name (Google CSE / scrape / Bing / social helpers). Optionally encode/match faces with `face_recognition`. Scrapes OG/meta avatar URLs from profile pages (used during username processing). Computes overall identity probability.

Key API: `search_images_by_name`, `extract_profile_image_from_url`, `extract_face_encoding`, `match_faces`, `search_and_match_identity`, `get_discovered_artifacts`.

Native safety: dlib is neither thread-safe nor tolerant of the buffer it is handed, and scraped candidates include palette, grayscale and 16-bit images. Every call into it is serialised by `_FACE_LOCK` and given a contiguous `uint8` RGB array; without both the process aborts with `free(): invalid size` mid-investigation.

`src/modules/leakosint.py`

Purpose: Keyed breach-record lookup against `leakosintapi.com`, whose records lead the report when present.

How it works: Token from the `leakosint` plugin config or `LEAKOSINT_API_KEY`. The API rejects a `limit` outside 100–10000 and temporarily blocks the token for it, so the request limit is clamped to `MIN_LIMIT` / `MAX_LIMIT`. Requests are throttled to one per `MIN_REQUEST_INTERVAL`. Non-200 replies keep the body's explanation rather than reporting a bare status code, and every failure path (transport, non-JSON, unexpected shape, API error) logs the query and reason — the token never appears in a log, report or database row.

Key API: `LeakRecord`, `Leak0sintResult`, `get_settings`, `get_api_token`, `is_configured`, `search`.

`src/modules/google_dorks.py`

Purpose: Username/name discovery via Google dork patterns (CSE API or DuckDuckGo/Google/Bing scrape).

How it works: Builds `site:` / `inurl:` dorks → rate-limits process-wide → caches results → parses SERPs into platform/username/email-like artifacts. Parallel pattern execution. Availability is true if API key present or scraping allowed.

Key API: `GoogleDorksSearch`, `search_username`, `run_google_dorks_search`, `check_google_dorks_availability`.

`src/modules/external_tools.py`

Purpose: Unified subprocess/API integrations for installed OSINT CLIs and Wayback CDX.

Integrated tools: Sherlock, Maigret, Holehe, OSRFramework, theHarvester, Subfinder, Sublist3r, WhatWeb, Shodan, Amass, Whois, Nmap, ExifTool, Wayback Machine. (`dig` is deliberately not dispatched: for an email seed it profiles the mail provider, not the subject.)

How it works:

1. Each `*Integration` class builds a CLI (or HTTP for Wayback), runs it under timeout inside `io_slot`, and hands the output to that tool's parser in `tool_parsers.py` (capped at `MAX_ARTIFACTS_PER_TOOL=15`).
2. `@skip_if_not_available` short-circuits missing tools.

3. `run_tool_analysis(tool, analysis_type, target)` dispatches + memoizes on `(tool, analysis, target)` (cache cleared per investigation).
4. Orchestrator selects analysis by artifact type (username → sherlock/maigret/...; domain → whois/enum; ip → nmap/shodan; email → holehe; image → exiftool).
5. Structured output (Maigret NDJSON, WhatWeb JSON, Nmap XML) falls back to stdout parsing when the file is absent, malformed, or yields nothing.

Subprocess safety (`_run_subprocess` / `_terminate` / `_decode`): `subprocess.run(timeout=...)` kills only the process it started, so a tool that forks (amass, theHarvester) left children holding the pipes and the cleanup read blocked forever. Each tool now leads its own process group (`start_new_session=True`) and the group is signalled as a whole (SIGTERM, then SIGKILL after 5s), so the deadline is always enforced. What a tool printed before its deadline is kept, output is decoded with `errors="replace"`, and capped at `MAX_TOOL_OUTPUT_BYTES` (32 MB) since it is held in memory. Every failure stays inside the returned `ToolResult`.

Key maps: `ANALYSIS_METHODS`, `TOOL_ARTIFACT_TYPES`, `UNIMPLEMENTED_TOOLS`, `get_tool_integrations`, `get_tool_coverage`.

Output artifact types include: `username_presence`, `email_presence`, `subdomain`, `open_port`, `historical_url`, `web_technology`, `domain_info`, `gps_coordinates`, ...

[src/modules/tool_parsers.py](#)

Purpose: One parser per tool, reading that tool's own output format. Previously a single sherlock-shaped regex was applied to every tool, which under-collected badly.

Parser	Reads	Recovers
<code>parse_sherlock</code>	[+] Platform: url lines	<code>username_presence</code>
<code>parse_maigret_ndjson</code>	Maigret NDJSON report	platform, URL, plus fullname / image / location / bio / follower counts from site extractors
<code>parse_usufy_profiles</code>	OSRFramework JSON	<code>username_presence</code>
<code>parse_holehe_csv</code> / <code>parse_holehe_text</code>	CSV report, else stdout	<code>email_presence</code>
<code>parse_theharvester_json</code>	theHarvester JSON	emails, hosts, IPs, URLs, ASNs — contacts and host/network capped independently , so a large email set cannot suppress subdomains
<code>parse_subfinder_json</code> / <code>parse_subdomains</code>	JSONL, else plain lines	<code>subdomain</code>
<code>parse_whois</code>	Key/value text	registrar, dates, org, emails, and every repeated label (all name servers, not just the first)
<code>parse_whatweb_json</code> / <code>parse_whatweb_summary</code>	<code>--log-json</code> , else stdout	<code>web_technology</code> , <code>ip_address</code>
<code>parse_nmap_xml</code> / <code>parse_nmap_text</code>	XML report, else stdout	<code>open_port</code> with service/product
<code>parse_exiftool_json</code>	ExifTool JSON	GPS, camera, creation dates, identity tags
<code>parse_shodan_host</code>	Shodan host JSON	ports, services, hostnames (list or string form, without Python list punctuation)
<code>parse_wayback_cdx</code>	CDX rows	<code>historical_url</code>

[src/modules/correlation.py](#)

Purpose: Lightweight in-memory NetworkX correlation metrics (no DB persona building) used at orchestrator finalize.

How it works: Builds graph from artifact/link lists → counts components → scores multi-node components → extracts risk from metadata JSON → serializes to `investigation_metadata` key `correlation_analysis`. Full persona building lives in `correlation/linker.py`.

[src/modules/correlation_neo4j.py](#)

Purpose: Optional Neo4j-backed correlation with Cypher component analysis and cross-investigation queries.

How it works: Bolt connect → ensure constraints → clear/rebuild investigation subgraph → Cypher for components/clusters/metrics/risks. Can correlate across investigation IDs. Orchestrator falls back to NetworkX on errors.

Config: `neo4j_uri` / `user` / `password` / `database` on `InvestigationConfig`.

6. Plugin subsystem

`src/plugins/__init__.py`

Public exports: `OSINTPlugin`, `PluginResult`, `PluginConfig`, `Artifact`, `PluginManager`, `PluginRegistry`.

`src/plugins/base.py`

Purpose: Abstract plugin contract and shared DTOs.

Type	Role
<code>PluginStatus</code>	<code>success</code> / <code>failure</code> / <code>partial</code> / <code>skipped</code>
<code>PluginConfig</code>	<code>enabled</code> , <code>timeout</code> , <code>max_retries</code> , <code>api_key</code> , <code>custom_params</code>
<code>Artifact</code>	<code>type</code> , <code>value</code> , <code>source</code> , <code>confidence</code> , <code>metadata</code>
<code>PluginResult</code>	Execution outcome + findings list
<code>OSINTPlugin</code> <code>ABC</code>	<code>get_name/version/description</code> , <code>get_supported_artifact_types</code> , <code>is_available</code> , <code>execute</code> , plus <code>validate/pre/post hooks</code>

`src/plugins/registry.py`

Purpose: Discover, register, and instantiate plugin classes from `builtins/`.

How it works: Imports each `src.plugins.builtins.*.py` (skips `_`-prefixed), finds concrete `OSINTPlugin` subclasses, registers by **class name**. Lazily instantiates. Filters by `is_available()`. Global registry via `get_global_registry`.

`src/plugins/manager.py`

Purpose: Execute plugins (serial or parallel) and aggregate findings.

How it works: Reads per-plugin enabled/timeout from config → skips disabled/unavailable/unsupported types → optionally `ThreadPoolExecutor` for compatible plugins → tracks success/fail/skip timing stats.

Key API: `execute_plugin`, `execute_plugins_for_artifact`, `execute_plugins_for_artifacts`, `aggregate_findings`, `get_execution_stats`.

`src/plugins/integration_plugin.py`

Purpose: Thin adapter so builtins can declare `tool_name` / `analysis_type` / `artifact_types` and delegate to `run_tool_analysis` without duplicating parsers.

How it works: Availability via `check_tool_availability` (or always true if HTTP-only). Success path converts each discovered dict to `Artifact`, stashing non-core keys in metadata. `check_wiring` validates analysis exists in `ANALYSIS_METHODS`.

7. Built-in plugins

`src/plugins/builtins/__init__.py`

Re-exports 19 plugin classes in `__all__`. Note: `ImageMatchPlugin` and `LeakosintPlugin` exist on disk but are **not** exported here (the registry still discovers them via filesystem import).

Module-backed plugins (wrap `src/modules/*`)

File	Class	Accepts	Wraps	Produces
username_search_plugin.py	UsernameSearchPlugin	username	username_search.search_username	platform_presence
email_breach_plugin.py	EmailBreachPlugin	email	email_osint.analyze_email / HIBP	breach
phone_validation_plugin.py	PhoneValidationPlugin	phone	phone_osint.analyze_phone	location, carrier
google_dorks_plugin.py	GoogleDorksPlugin	username	google_dorks.run_google_dorks_search	username, email, domain
image_match_plugin.py	ImageMatchPlugin	fullname	image_match + optional face_recognition	image_url, face_match
leakosint_plugin.py	LeakosintPlugin	email, phone, username, fullname	leakosint.search	leak_record (sorted to the top of the report)

Direct CLI / HTTP plugins

File	Class	Accepts	Tool	How	Produces
sherlock_plugin.py	SherlockPlugin	username	Sherlock	sherlock ... --format json ; keep status==found	platform_presence
theharvester_plugin.py	TheHarvesterPlugin	domain	theHarvester	-b google JSON/text parse	email, domain
shodan_plugin.py	ShodanPlugin	ip_address, domain	Shodan	API key required; host/domain search	host_info, open_port / service
whois_plugin.py	WhoisPlugin	domain, ip_address	whois	Line-scan registrant/org/email	organization, email
profile_image_plugin.py	ProfileImagePlugin	platform_presence	HTTP + BeautifulSoup	CSS/og:image/twitter:image heuristics	image

IntegrationPlugin wrappers (delegate to `external_tools`)

File	Class	tool_name / analysis	Accepts	Produces
maigret_plugin.py	MaigretPlugin	maigret / username_search	username	username_presence
holehe_plugin.py	HolehePlugin	holehe / email_check	email	email_presence
subfinder_plugin.py	SubfinderPlugin	subfinder / subdomain_enum	domain	subdomain
sublist3r_plugin.py	Sublist3rPlugin	sublist3r / subdomain_enum	domain	subdomain
amass_plugin.py	AmassPlugin	amass / subdomain_enum	domain	subdomain
whatweb_plugin.py	WhatWebPlugin	whatweb / tech_fingerprint	domain, subdomain	ip_address, web_technology
nmap_plugin.py	NmapPlugin	nmap / host_scan	ip_address	open_port
exiftool_plugin.py	ExifToolPlugin	exiftool / metadata_extract	image (local path)	gps_coordinates, camera_info, creation_date
wayback_plugin.py	WaybackMachinePlugin	wayback_machine / historical_urls	domain	historical_url
osrframework_plugin.py	OsrframeworkPlugin	osrframework / username_search	username	username_presence

Several tools exist both as orchestrator `external_tools` integrations **and** as plugins. Artifact type names can differ (`platform_presence` vs `username_presence`).

8. Correlation

`src/correlation/__init__.py`

Package docstring only.

`src/correlation/linker.py`

Purpose: Build filtered NetworkX identity graph, extract connected-component profiles, attach external-tool findings.

How it works:

1. Load artifacts/links/presences from DB.
2. Filter to identity types with confidence ≥ 0.3 and noise rejection (`NOISE_PATTERNS`).
3. Undirected connected components $\rightarrow$ `IDENTITY-MNN` profiles (`phones`, `emails`, `usernames`, `images`, `platforms`).
4. Weak/noisy components $\rightarrow$ `IDENTITY-NOISE`.
5. Tool artifacts (subdomains, ports, historical URLs, account presences) are **excluded** from the identity graph but **attached** by expanding from each component over the full link graph, stopping at foreign identity nodes.
6. Profiles sorted by artifact count; confidence from type diversity + cross-type edges + avg edge confidence.

Key types: `IdentityProfile`, `CorrelationResult`, `build_identity_graph`, `correlate_identities`, `_attach_tool_findings`, `_compute_confidence`.

`src/correlation/scorer.py`

Purpose: Link confidence and identity risk scoring / classification.

How it works:

- Link score = `BASE_SCORES[link_type]` $\times$ freshness decay ($>2y \times 0.6$, $>1y \times 0.8$) $\times$ source reliability, capped at 1.0.
- Risk score sums weighted indicators (VoIP, disposable email, breaches, ...), with dynamic `found_in_N_breaches` handling.
- Levels: ≥ 0.8 critical, ≥ 0.6 high, ≥ 0.4 medium, ≥ 0.2 low, else minimal.

Key API: `compute_link_confidence`, `compute_identity_risk_score`, `classify_risk_level`.

9. Reporting and visualization

`src/reporting/__init__.py`

Package docstring only.

`src/reporting/html_report.py`

Purpose: Jinja2 HTML (4 templates) and JSON investigation reports.

Templates: `standard` (loaded from `templates/standard.html`), plus in-module `EXECUTIVE_TEMPLATE`, `TECHNICAL_TEMPLATE`, `LEGAL_TEMPLATE` strings rendered through a Jinja2 `BaseLoader`.

How it works:

1. Load investigation + artifacts/links/presences.
2. Run `correlate_identities`; score each identity via scorer.
3. Build derived views: timeline, key findings, confidence metrics, risk matrix, platform heatmap, anomalies, tool metrics, recommendations, priority queue, geographic data, verification status, auto-escalation — plus the `report_data`

- sections (leaks, per-artifact detail, evidence, citations, cross-investigation hits, run-to-run delta).
- 4. Embed a navigable pyvis graph snippet; inline or link images, skipping known placeholder avatars.
- 5. Render selected template → write `reports/{INV}_report.html` (or custom path).
- 6. `generate_json_report` mirrors `meta/summary/identities/artifacts/links/presences/graph/tool_metrics`.

Redaction order matters: cross-investigation matching queries the database with the `stored` values (`stored_artifacts`) and masks only the hits it returns — masking first would look up `[REDACTED]` and silently lose the section.

Key helpers: `_select_template` , `_build_artifact_views` , `_confidence_basis` , `_build_identity_artifacts` , `_prepare_graph_embed_html` , `_resolve_image_src` .

Called by: CLI `investigate` / `report` , workflow API.

`src/reporting/templates/standard.html`

The default report template, kept as a file rather than a string so its CSS is editable and diffable. Carries the light/dark branding rules, the section layout, per-artifact drill-downs (`<details>`) and the responsive tables. Every surface has an explicit colour — the template previously inherited the reader's scheme, which rendered text on same-coloured backgrounds.

`src/reporting/report_data.py`

Purpose: The derived sections of a report and, importantly, the redaction that shareable copies depend on.

Sections built here: `DEFAULT_SECTIONS` = leaks, identities, summary, tools, platforms, graph, artifacts, evidence, orphans, geo, audit, comments, cross, delta, filters. `parse_sections` resolves `--sections` .

Function	Role
<code>build_timeline</code> / <code>parse_event_date</code>	Plot <i>when things happened</i> — breach, EXIF, Wayback and registration dates — not when they were found; Wayback <code>YYYYMMDDhhmmss</code> , EXIF <code>YYYY:MM:DD</code> and ISO forms all parsed
<code>build_preserved_evidence</code>	Re-hash captures and report verified/modified/missing
<code>build_source_citations</code>	The exact command, run time and tool version behind each artifact
<code>build_evidence_chains</code>	Discovery path from seed to artifact
<code>build_leak_findings</code>	Leak records, ordered, reduced to their database name when masked
<code>build_orphan_findings</code> , <code>build_actionable_recommendations</code> , <code>build_cross_investigation</code> , <code>find_previous_investigation</code>	Remaining sections
<code>enrich_tool_status</code>	Distinguishes productive / silent / unavailable / failed tools, counting presences as output rather than only artifacts

Redaction: `REDACT_TYPES` masks phone, email, image, fullname, GPS and location; `REDACT_URL_TYPES` replaces avatar and profile URLs with `[REDACTED_URL]` , since masking characters out of a URL leaves the handle standing; platform biographies are redacted too, and masking recurses through nested metadata rather than only top-level values.

`src/reporting/exports.py`

Purpose: Non-HTML outputs. `export_artifacts_csv` / `export_presences_csv` flatten rows to CSV; `generate_pdf_from_html` renders a report through WeasyPrint, with `_simplify_html_for_pdf` stripping the interactive graph and dark styling that do not survive print.

`src/graph/__init__.py`

Package docstring only.

`src/graph/visualizer.py`

Purpose: Interactive pyvis HTML identity network + graph statistics.

How it works: `build_identity_graph` → map nodes to colored/shaped pyvis nodes (`TYPE_COLORS` / `TYPE_SIZES` / `TYPE_SHAPES`) with tooltips → edges from links → standalone HTML. `get_graph_stats` returns nodes, edges, components, density, type distribution.

Collections: same-type leaf neighbours are collapsed into one labelled count node once they reach `DEFAULT_COLLECTION_THRESHOLD` (8, 0 disables), which keeps a large investigation legible; `build_collections` decides, `_interaction_script` expands one on click. `LAYOUTS` offers `organic`, `hierarchical` and `circular`, and nodes link through to their detail section in the report.

Config: `graph.{layout,collection_threshold}`.

Called by: CLI `graph`, `html_report` embed helper.

10. Utilities

`src/utils/__init__.py`

Re-exports tool-checker API: `ToolChecker`, `ToolInfo`, `ToolStatus`, `get_tool_checker`, `check_tool_availability`, `skip_if_not_available`.

`src/utils/concurrency.py`

Purpose: Global semaphore capping concurrent leaf I/O across nested thread pools.

How it works: `BoundedSemaphore` sized from `investigation.max_concurrent_io` (default 32). Leaf HTTP/subprocess code acquires via `io_slot` context manager. Coordinating tasks must **not** hold slots while waiting on children (avoids deadlock). Orchestrator calls `configure` at start.

`src/utils/http_client.py`

Purpose: Shared pooled `requests.Session` with retries, UA rotation, rate limit, adaptive timeout, and `io_slot` bounding.

How it works: Lazy `_BoundedSession` with large connection pool, urllib3 Retry (5xx only), Accept-Encoding limited to urllib3-supported decoders. Optional adaptive timeout from recent latency. Min interval between requests.

Config: `http_client`.

`{timeout,min/max_timeout,adaptive_timeout,connection_pool_size,retry_*,min_request_interval,user_agents}`.

`src/utils/tool_checker.py`

Purpose: Detect which external OSINT CLIs/APIs are available on the host.

How it works: Registers ~30-35 known tools; `shutil.which` + optional `--version` (memoized once per process under a lock). API-based tools (`wayback_machine`, `google_dorks`) marked available without a binary. `@skip_if_not_available` decorator returns `None` if missing.

Key API: `check_tool`, `check_all_tools`, `is_available`, `get_available_tools`, `get_missing_tools`, `print_status`, `get_tool_checker`.

`src/utils/matching.py`

Purpose: Decide whether a tool's finding really belongs to the target, so a partial or substring match is not attributed to them.

How it works: `MatchPolicy` (from `investigation.strict_match`, overridable with `--strict-match/--no-strict-match`) requires the target's exact handle to appear on a token boundary — `contains_exact` — or as a host label in a URL. `filter_full_matches` applies this to account-type findings (`username_presence`, `email_presence`, `platform_presence`); dropped counts are logged per artifact.

11. Analysis and collaboration

`src/analysis/pattern_recognition.py`

Purpose: Cross-investigation pattern mining (recurring artifacts/platforms/risks, pairwise similarity).

How it works: Load all investigations → counters for artifacts/platforms/risks → Jaccard-like pairwise similarity. Standalone analytics (not on main investigate path).

`src/analysis/statistical_analysis.py`

Purpose: Generic stats helpers (CI, t-test, sample size, distribution moments) using stdlib `statistics` / `math`. Not wired into CLI investigate.

`src/analysis/trend_analysis.py`

Purpose: Compare recent investigation metrics to a historical baseline window.

How it works: Aggregate counts for baseline vs analysis periods → % change / significance → emerging-threat flags. Params: `baseline_days`, `analysis_days`.

`src/collaboration/comments.py`

Purpose: Threaded comments/annotations on investigations and artifacts.

How it works: Ensures `comments` table exists; CRUD with optional `parent_id` threading and `comment_type`. Not on the main CLI path.

12. API

`src/api/workflow_api.py`

Purpose: Flask REST wrapper around investigation create/list/get/report/artifacts/links/risk.

Endpoints (under `/api/v1/`): `/health`, `/investigations` (POST/GET), `/investigations/<id>`, `/report`, `/artifacts`, `/links`, `/risk`.

How it works: CORS Flask app; POST runs `run_investigation`; GET endpoints read DB / generate reports. Optional `X-API-Key` auth against in-memory key set. Factory: `create_api_server(host, port, debug)`.

See [ARCHITECTURE.md §8](#) for known API drift vs current orchestrator/DB field names.

13. Config, scripts, packaging, Docker

config/config.yaml

Section	Purpose
plugins	Per-plugin <code>enabled</code> , <code>priority</code> , <code>timeout</code> , <code>max_retries</code> , optional <code>api_key</code>
plugin_settings	Parallelism, caching, rate limit
investigation	<code>max_depth</code> , <code>max_runtime_minutes</code> , <code>max_total_artifacts</code> , <code>max_concurrent_io</code> , <code>strict_match</code> , feature flags, Neo4j
reporting	Auto-generate, format, <code>output_dir</code> , template, sections, redact
graph	Default layout and <code>collection_threshold</code>
evidence	Preservation on/off, capture directory, capture size cap
database	SQLite path, backup
http_client	Timeouts, pool, retries, user agents
google_dorks	Pattern/rate limits, scrapers
username_search	Workers + per-platform check definitions
email_osint	Disposable/privacy domain lists
breach_check	HIBP base URL + rate limit
orchestrator	BFS batch size, depth, parallel workers
investigation.strict_match	<code>enabled</code> , <code>require_validated_presence</code> , name-variant expansion

config/docker-compose.yml

Builds `Dockerfile` → `ghost-identity-hunter:latest`. Service `ghost-hunter` mounts DB volume + investigations; entrypoint keeps container alive (`tail -f /dev/null`).

config/docker-compose.kali.yml

Builds `Dockerfile.kali` → Kali image. Services: `ghost-hunter-kali` (interactive TTY, Neo4j env) + `neo4j:5.15.0-community` (7474/7687, APOC). Shared network and volumes.

Dockerfile

`python:3.11-slim` → install deps → copy `src/` → non-root `ghosthunter` → ENTRYPOINT `python src/cli.py`.

Dockerfile.kali

`kalilinux/kali-rolling` → apt OSINT tools (nmap, whois, dig, whatweb, exiftool, ...) → Python venv + editable install → `CMD` `bash`.

scripts/run.sh

Creates/activates `venv` if missing; installs editable package if needed; runs `python -m src.cli "$@"` from repo root.

scripts/run.py

Cross-platform Python launcher; warns if not in venv; invokes `src.cli.cli()` after adding project root to `sys.path`.

pyproject.toml

Package `ghost-identity-hunter` 0.1.0 (Python ≥3.10). Console script: `ghost-hunter = src.cli:cli`. Core deps: click, networkx, phonenumbers, requests, bs4, Pillow, pyvis, Jinja2, tabulate. Optional `dev`: pytest, pytest-cov, ruff; optional `face`:

face_recognition plus `setuptools<81`. The build requirement is capped the same way — `face_recognition_models` still imports `pkg_resources`, which setuptools 81 removed, so an uncapped install fails on Python 3.13. Lift both once that package is fixed upstream.

`requirements.txt` / `requirements-optional.txt`

`requirements.txt` is the runtime install list, including extras vs pyproject: Brotli, neo4j, pyyaml, numpy, urllib3 (plus pytest/ruff for local use). `requirements-optional.txt` holds the face-matching extra and its `setuptools<81` cap.

14. Tests

File	Purpose
test_storage.py	SQLite CRUD: investigations, artifacts, links, platform presence
test_orchestrator.py	BFS pipeline, bounds, artifact dispatch, rediscovery linking
test_correlation.py	Confidence/risk scoring and identity correlation
test_cli_seeds.py	CLI options → seed artifacts; report output paths
test_cli_full_name.py	--full-name / -n seed → fullname artifact
test_fullname_coverage.py	Full-name → username candidates, image parsing, encodings
test_username_search.py	Platform web/API checks, redirects, discovered artifacts
test_email_osint.py	Email validation and analysis
test_phone_osint.py	Phone analysis
test_image_search.py	Image OSINT analysis and file hashes
test_google_dorks_filter.py	Junk username filter / extractor skip rules
test_external_tools.py	Parser coverage + correlation of tool artifacts
test_integration_plugins.py	IntegrationPlugin wiring, registry, execute, usufy parser
test_html_report.py	HTML/JSON reports, templates, tool metrics
test_perf_tier1.py	I/O semaphore, tool-analysis memoization, rate-limit concurrency
test_tool_checker_cache.py	ToolChecker per-process memoization
test_tool_parsers.py	Each tool's own output format, including malformed structured reports falling back to stdout
test_resilience.py	Real subprocesses: a tool that never returns, a forking tool (and that its grandchild is gone), invalid UTF-8, unbounded output, a missing binary, correlation failure, and an aborted run that still reports
test_face_encoding.py	What dlib is handed for palette/grayscale/RGBA/16-bit input, and that threads never reach it together
test_redaction.py	Personal detail, biographies and profile/avatar URLs masked everywhere; cross-investigation matching still works under --redact
test_evidence.py	Capture, hashing, truncation, permissions, verification
test_timeline.py	Event-date parsing across Wayback/EXIF/ISO forms
test_delta.py	Run-to-run comparison, including masked leak records
test_source_citations.py	Command/version/run attribution per artifact
test_confidence_provenance.py	Why a score is what it is
test_graph_visualizer.py	Layouts, collections, node/edge mapping
test_leakosint.py	Limit clamping, failure logging, token never emitted
test_strict_match.py	Exact-handle matching and dropped partial findings
test_cli_seeds.py / test_cli_full_name.py	CLI seed handling and report paths

Related documents

- [ARCHITECTURE.md](#) — layered system view, ER diagram, known gaps
- [LLD.md](#) — low-level design per subsystem
- [SEQUENCE_DIAGRAMS.md](#) — end-to-end runtime sequences
- [plugin_development.md](#) — writing new plugins
- [TOOL_COVERAGE.md](#) — which tools are detected vs executed